PRÁCTICA OBLIGATORIA - OPENCV

Unai García Blanco - [redacted]
Jorge Caballero González - [redacted]

4º Grado de Ingeniería Informática - USAL

Introducción:

En esta práctica realizaremos un robot que va navegando por un entorno de oficinas con una cámara acoplada e irá buscando y leyendo códigos BIDI embedidos en un cuadrado rojo.

Para la solución usaremos la visión artificial con la biblioteca OpenCV y elegimos Python como lenguaje de programación para realizar la práctica, y además usaremos las bibliotecas numpy, os y random de Python para añadir algunas funcionalidades al programa.

Funcionamiento:

Dividimos el programa en 10 métodos distintos:

- calcularVerticeAbajoDer(r):
  Obtiene la posición de la esquina inferior derecha del código BIDI

- calcularDesplazamientoHorizontal(ai, as, src):
  Calcula el desplazamiento del centro de la cámara del robot respecto del centro del código BIDI, indicándonos hacia dónde estamos desplazados en un eje x cartesiano.

- calcularDesplazamientoVertical(ai, as, src):
  Calcula el desplazamiento del centro de la cámara del robot respecto del centro del código BIDI, indicándonos hacia dónde estamos desplazados en un eje y cartesiano.

- traducirVertical(d):
  A partir de la distancia calculada en calcularDesplazamientoVertical(), calcularemos si está dentro de un rango de error asumible para determinar si está en una posición correcta respecto al eje y cartesiano, si tiene que moverse el robot hacia arriba o hacia abajo (eje y)

- traducirHorizontal(d):
  A partir de la distancia calculada en calcularDesplazamientoHorizontal(), calcularemos si está dentro de un rango de error asumible para determinar si está en una posición correcta respecto al eje y cartesiano, si tiene que moverse el robot hacia arriba o hacia abajo (eje y)

- calcularTamano(r, src):
  Calcula si el tamaño del código BIDI está dentro de un rango aceptable respecto al tamaño de la imagen recogida por la cámara.

- traducirTamano(r, src):
  Calcula si el tamaño del código BIDI está dentro de un rango aceptable respecto al tamaño de la imagen recogida por la cámara, pero devuelve un mensaje con la orden que debería realizar el robot para adaptar el tamaño del BIDI.

- imprimirResultados(contour, src):
  Imprime en la terminal todos los resultados calculados y recogidos por el resto de métodos.

- cargar_imagenes():
  Carga las imágenes de prueba desde la carpeta images situada en la misma carpeta que el .py

- main():
  Carga el array de imágenes y toma un índice aleatorio para que cada vez que se ejecute el programa, se muestre una imagen distinta en una ventana, además de su conversión a gris.
  Después, convertirá la imagen a grises para obtener la información de manera más precisa y usará cv2.findContours() para obtener los bordes del código BIDI y así facilitar los cálculos de posicionamiento, ya que le pasaremos la información del contorno al método imprimirResultados().
  Le hemos añadido un pequeño cartel con los controles, simulando que fueran los controles de movimiento del robot, y se manejará con las teclas w, a, s, d, e y r para obtener los resultados esperados. El funcionamiento consiste en que según “controlemos” el movimiento, se cambiará el índice del array de imágenes y se mostrará otra imagen.
  Se saldrá del programa con la tecla Q.

Ejemplo de ejecución:



